Politechnika Śląska
Wydział Automatyki, Elektroniki i Informatyki
Informatyka, studia inżynierska, rok 2, semestr 4

NEURAL NETWORK
Hand Written Digit Recognization

Autor: Szymon Kwieciński
Indeks: 312140

Spis treści

WPROWADZENIE **3**

Opis Tematu Projektu 3
Cele Projektu 3

ARCHITEKTURA SYSTEMU I UŻYTE TECHNOLOGIE **4**

Ogólny diagram klas 4
Wykorzystane Technologie i Biblioteki 5

IMPLEMENTACJA KLAS **6**

Klasa Matrix<T> 6
Klasa Activation<T> 6
Klasa LossFunction<T> 6
Klasa DataLoader<T> 7
Klasa Layer<T> 7
Klasa NeuralNetwork<T> 8
Klasa Trainer<T> 9
Klasa Metrics<T> 9
Klasa User i UserManager 9
Klasa Utils i Globals 9

INTERFEJS UŻYTKOWNIKA **10**

Interfejs Konsolowy (Panel Sterowania) 10
Interfejs Graficzny (SFML Mode) 10

WYZWANIA I PROBLEMY NAPOTKANE PODCZAS PROJEKTU **11**

Typowe Błędy i Problemy 11
Problemy ze Wsparciem dla Modułów C++20 12

UZYSKANA WIEDZA I DOŚWIADCZENIE **13**

PODSUMOWANIE **14**

MOŻLIWOŚĆ DALSZEGO ROZWOJU **15**

Implementacja Zaawansowanych Optymalizatorów 15
Techniki Regularyzacji 15
Wsparcie dla Innych Typów Warstw 15
Ulepszone Zarządzanie Pamięcią GPU i Asynchroniczność 15
Serializacja i Deserializacja Całej Sieci (Rozszerzenie) 16
Wsparcie dla Innych Typów Danych i Zbiory Danych 16
Zwiększona Odporność na Błędy i Logowanie 16
Wizualizacje w Czasie Rzeczywistym 16
Ulepszone Zarządzanie Użytkownikami i Bezpieczeństwo 16
Pakowanie w Uniwersalną Bibliotekę i Interfejsy dla Innych Języków 16
Wdrożenie Testów Jednostkowych i Mutacyjnych 16
Integracja z Uczenia ze Wzmocnieniem (Reinforcement Learning) do Zastosowań w Grach 17

LINKI DO UŻYWANYCH BIBLIOTEK **18**

Wprowadzenie

Opis Tematu Projektu

Projekt polegał na implementacji od podstaw, w języku C++, pełnowymiarowej, wielowarstwowej sieci neuronowej (Multi-Layer Perceptron - MLP), specjalnie przystosowanej do zadania rozpoznawania cyfr pisanych ręcznie, z wykorzystaniem powszechnie znanego zbioru danych MNIST. Celem było nie tylko stworzenie działającego modelu, ale również dogłębne zrozumienie i praktyczna implementacja fundamentalnych algorytmów uczenia maszynowego oraz optymalizacja wydajności. Zintegrowano również interaktywne interfejsy – konsolowy panel sterowania oraz graficzny tryb rysowania w SFML – w celu ułatwienia interakcji z modelem, jego konfiguracji, treningu i oceny. System wzbogacono o mechanizmy zarządzania użytkownikami, trwały zapis stanu sieci oraz wizualizację wyników treningu.

Cele Projektu

Główne cele, które kierowały rozwojem projektu, obejmowały:

- **Implementacja podstaw:** Stworzenie klas dla operacji algebry liniowej na macierzach (`Matrix`), funkcji aktywacji (`Activation`), funkcji straty (`LossFunction`) oraz zarządzania danymi (`DataLoader`).
- **Budowa architektury sieci:** Zaimplementowanie klas `Layer` i `NeuralNetwork` do konstrukcji i zarządzania strukturą sieci.
- **Algorytm uczenia:** Realizacja algorytmu propagacji wstecznej (`backpropagation`) do efektywnego trenowania sieci.
- **Wykorzystanie danych:** Adaptacja zbioru danych MNIST do potrzeb projektu, włącznie z ładowaniem danych binarnych i ich preprocessingiem.
- **Optymalizacja wydajności:** Wykorzystanie współbieżności CPU (wątki `std::thread`) oraz akceleracji GPU (NVIDIA CUDA i biblioteki cuBLAS) do przyspieszenia kluczowych operacji macierzowych.
- **Interaktywność:** Stworzenie konsolowego interfejsu użytkownika do konfiguracji, treningu i monitorowania sieci.
- **Wizualizacja graficzna:** Implementacja trybu graficznego SFML umożliwiającego rysowanie cyfr i ich rozpoznawanie w czasie rzeczywistym.
- **Trwałość:** Dodanie funkcjonalności zapisu i odczytu konfiguracji sieci (JSON) oraz wytrenowanych wag i biasów (CSV), z personalizacją dla poszczególnych użytkowników.
- **Raportowanie:** Generowanie wykresów historii treningu za pomocą zewnętrznego narzędzia Gnuplot.

Architektura Systemu i Użyte Technologie

Ogólny diagram klas

System został zaprojektowany z myślą o modułowości, gdzie każda klasa odpowiada za ściśle określoną część funkcjonalności. Poniżej przedstawiono schemat kluczowych klas i ich relacji.

- **`main.cpp`**: Działa jako główny punkt wejścia i orkiestrator. Zawiera pętlę menu konsoli, tworzy i zarządza instancjami `UserManager`, `NeuralNetwork`, `DataLoader`, `Trainer` oraz wywołuje funkcje `Utils`.
- **`UserManager`**: Agreguje obiekty `User`. Odpowiada za zarządzanie kontami (rejestracja, logowanie) i mapowanie użytkowników na ich katalogi modeli.
  - **`User`**: (Struktura danych) Reprezentuje użytkownika z `username` i `password`.
- **`NetworkConfig`**: (Struktura danych) Przechowuje pełną konfigurację sieci (rozmiary wejścia/wyjścia, learning rate, funkcja straty, optymalizator, tryb mnożenia, lista `LayerConfig`). Posiada statyczne metody `toJson()` i `fromJson()` do serializacji/deserializacji.
  - **`LayerConfig`**: (Struktura danych) Przechowuje konfigurację pojedynczej warstwy (rozmiar wyjścia, nazwa funkcji aktywacji).
- **`NeuralNetwork<T>`**: Reprezentuje całą sieć neuronową. Agreguje `std::vector<std::unique_ptr<Layer<T>>>` (lista warstw) oraz obiekty `LossFunction` i `NetworkConfig`. Jest to fasada dla operacji na warstwach.
- **`Layer<T>`**: Reprezentuje pojedynczą warstwę sieci. Agreguje `Matrix<T>` (dla wag, biasów, gradientów, aktywacji) oraz `Activation<T>`.
- **`Matrix<T>`**: Podstawowa klasa dla operacji algebry liniowej. Przechowuje dane w jednowymiarowym `std::vector<T>`(row-major order).
  - **Statyczne:** `MultiplicationMode currentMultiplicationMode` (wybór CPU/CUDA) oraz metody `setMultiplicationMode()`, `getCurrentMultiplicationMode()`.
- **`DataLoader<T>`**: Odpowiada za ładowanie i dostarczanie danych treningowych/testowych. Agreguje `Matrix<T>` dla obrazów i etykiet oraz `std::vector<size_t> sampleIndices` do tasowania.
- **`Trainer<T>`**: Koordynuje proces treningu. Posiada referencje do `NeuralNetwork` i `DataLoader`.
- **`Metrics<T>`**: (Klasa statyczna) - Oblicza metryki wydajności, takie jak dokładność.
- **`Activation<T>`**: Klasa zarządcza dla funkcji aktywacji (np. Sigmoid, ReLU, Softmax).
- **`LossFunction<T>`**: (Klasa abstrakcyjna) - Definiuje interfejs dla funkcji straty.
  - `MeanSquaredError<T>`: (Konkretna implementacja)
  - `CrossEntropyLoss<T>`: (Konkretna implementacja)
- **`CudaMatrixOps.h/.cu`**: Zawiera funkcje CUDA/cuBLAS (`extern "C"`) do akcelerowanego mnożenia macierzy i innych operacji na GPU.
- **`Utils.h/.cpp`**: Zbiór funkcji pomocniczych (np. UI konsoli, SFML, Gnuplot).
- **`Globals.h/.cpp`**: Definicje globalnych stałych konfiguracyjnych.

Wykorzystane Technologie i Biblioteki

- **C++20:** Język programowania. Wykorzystano nowoczesne cechy takie jak koncepty (`FloatingPoint`), `std::optional`, `std::thread`, `std::async`, `std::future`, `std::unique_ptr`, `std::filesystem`.
- **SFML 3.0 (Simple and Fast Multimedia Library):** Służy do tworzenia interaktywnego okna graficznego, rysowania cyfr, obsługi zdarzeń myszy/klawiatury i wyświetlania tekstu. Wybrano SFML 3.0 ze względu na prostotę biblioteki, dzięki czemu idealnie pasowała do projektu.
- **NVIDIA CUDA Toolkit:** Zestaw narzędzi deweloperskich i bibliotek NVIDII do programowania GPU. Pozwala na pisanie kodu (kerneli), który jest wykonywany bezpośrednio na procesorach graficznych NVIDII, co znacząco przyspiesza obliczenia równoległe.
- **NVIDIA cuBLAS:** Biblioteka Basic Linear Algebra Subprograms (BLAS) zoptymalizowana pod kątem GPU. Jest to kluczowa biblioteka do wysoce wydajnego mnożenia macierzy (GEMM) i innych operacji liniowo-algebraicznych na GPU.
- **nlohmann/json:** Nagłówkowa biblioteka C++ do łatwej pracy z formatem danych JSON. Wykorzystana do serializacji (zapisu) i deserializacji (odczytu) konfiguracji sieci.
- **Gnuplot:** Wolne, przenośne narzędzie do rysowania wykresów z linii komend. Używane do wizualizacji historii treningu (strat i dokładności), generując pliki PNG.
- **Standardowe biblioteki C++:** Szerokie wykorzystanie `<iostream>`, `<string>`, `<vector>`, `<limits>`, `<chrono>`, `<fstream>`, `<sstream>`, `<algorithm>`, `<random>`, `<iomanip>` i innych do zarządzania danymi, plikami, czasem, losowością i wyświetlaniem.

Implementacja Klas

Klasa Matrix<T>

- **Przeznaczenie:** Podstawowa jednostka dla wszystkich operacji algebry liniowej w sieci.
- **Kluczowe cechy:**
  - **Przechowywanie danych:** Używa `std::vector<T> data` w **row-major order** (wiersz-główny) dla optymalnej lokalności pamięci podręcznej CPU.
  - **Operatory:** Przeciążone operatory `+`, `-`, `*` (skalar), `*` (mnożenie macierzy) oraz metody `transpose()` i `hadamardProduct()`.
  - **Tryby optymalizacji:**
    - `currentMultiplicationMode`: Statyczna zmienna, która kontroluje, czy mnożenie macierzy (`operator*`) odbywa się na CPU czy GPU.
    - **Mnożenie macierzy (`operator*`):** Implementacja jest rozdzielona. Tryb `CPU_THREADS` używa `std::async` i `std::thread` do wielowątkowego obliczania iloczynu macierzy. Tryb `CUDA_GPU` wywołuje funkcję `cudaMatrixMultiplyCublas` z `CudaMatrixOps.cu`.
    - **Pozostałe operacje (`+`, `-`, `* skalar`, `hadamardProduct`, `applyFunction`, `transpose`):** Działają **zawsze na CPU**. `operator+`, `operator-`, `operator* scalar`, `hadamardProduct`, `applyFunction` wykorzystują `std::transform` (które jest kompilatorowo optymalizowane).
- **Wyzwania:** Zapewnienie, że `operator*` poprawnie przełącza się między CPU a GPU, oraz że alokacje i de-alokacje pamięci GPU są prawidłowo zarządzane nawet w przypadku wyjątków.

Klasa Activation<T>

- **Rola:** Wprowadza nieliniowość do sieci, niezbędną do modelowania złożonych wzorców.
- **Implementacja:** Klasa zarządzająca, która na podstawie nazwy (`std::string`) dostarcza metody `apply()` (funkcja aktywacji) i `derivative()` (jej pochodna).
- **Funkcje aktywacji:** Sigmoid, ReLU, Leaky ReLU, Tanh, Softmax. Implementacje tych funkcji są statycznymi metodami w `Activation.cpp`, wywoływanymi przez `applyFunction` (na CPU).

Klasa LossFunction<T>

- **Znaczenie:** Mierzy błąd (rozbieżność) między przewidywaniami sieci a prawdziwymi etykietami.
- **Implementacja:** Jest to klasa abstrakcyjna z metodami `calculateLoss()` i `calculateGradient()`.
- **Konkretne implementacje:**
  - `MeanSquaredError<T>` (MSE): Używana głównie w regresji.
  - `CrossEntropyLoss<T>`: Standard i najlepszy wybór dla klasyfikacji wieloklasowej (jak MNIST), zwłaszcza w połączeniu z Softmax.

Klasa DataLoader<T>

- **Przeznaczenie:** Efektywne wczytywanie i dostarczanie danych treningowych/testowych w postaci mini-batchy.
- **Ładowanie danych:**
  - `loadData()`: Koordynuje wczytywanie.
  - `loadImageData()` i `loadLabelData()`: Wczytują dane z binarnych plików MNIST (`.idx-ubyte`). Obsługują konwersję Big-Endian na Little-Endian.
  - **Optymalizacja:** Podczas wczytywania pikseli i etykiet, dane są najpierw wczytywane do buforów CPU (`std::vector<unsigned char>`), a następnie ich przetwarzanie (konwersja na `Matrix<T>`, one-hot encoding) jest **wielowątkowe (CPU)**, co znacząco przyspiesza fazę ładowania.
- **Dostarczanie batchy:**
  - `shuffle()`: Tasuje indeksy próbek (`sampleIndices`) na początku każdej epoki, aby randomizować kolejność danych.
  - `resetBatchIterator()`: Resetuje licznik batchy.
  - `nextBatch()`: Pobiera kolejny mini-batch obrazów i etykiet. Ta operacja jest również **wielowątkowa (CPU)**podczas kopiowania danych z głównych macierzy (`imageMatrix`, `labelMatrix`) do mniejszych macierzy batcha.

Klasa Layer<T>

- **Reprezentacja:** Pojedyncza warstwa sieci neuronowej.
- **Pola:** `weights`, `biases`, `weightsGradients`, `biasesGradients`, `preActivations` (Z), `activations` (A), `activationFunction`.
- **Metody:**
  - `initializeWeights()`: Inicjalizuje wagi losowymi wartościami z rozkładu normalnego (He initialization).
  - `forward(inputData)`:
    - `preActivations = inputData * weights;`: Mnożenie macierzy (może być GPU). Wynik jest na CPU.
    - `preActivations = preActivations + biases;`: Dodawanie biasów (wielowątkowe CPU).
    - `activations = activationFunction.apply(preActivations);`: Funkcja aktywacji (CPU).
  - `backward(inputDataForThisLayer, errorFromNextLayer)`:
    - `dActivation = activationFunction.derivative(preActivations);`: Pochodna aktywacji (CPU).
    - `currentLayerError = errorFromNextLayer.hadamardProduct(dActivation);`: Błąd warstwy (CPU).
    - `biasesGradients` (sumowanie): Obliczenie gradientów biasów jest **wielowątkowe CPU**.
    - `weightsGradients = inputDataForThisLayer.transpose() * currentLayerError;`: Obliczenie gradientów wag (mnożenie macierzy - może być GPU).

        - `errorToPreviousLayer = weights * currentLayerError.transpose();`: Obliczenie błędu do poprzedniej warstwy (mnożenie macierzy - może być GPU).
        - `errorToPreviousLayer = errorToPreviousLayer.transpose();`: Transpozycja (CPU).
    - `updateParameters(learningRate)`: Aktualizuje wagi i biasy za pomocą prostego SGD (na CPU).

Klasa NeuralNetwork<T>

- **Główna klasa:** Agreguje warstwy i zarządza całym cyklem sieci.
- **Pola:** `std::vector<std::unique_ptr<Layer<T>>> layers`, `learningRate`, `lossFunction`, `config` (obiekt `NetworkConfig`).
- **Konstruktory:**
    - Domyślny (`NeuralNetwork()`): Tworzy pusty obiekt, gotowy do wczytania konfiguracji.
    - `NeuralNetwork(const NetworkConfig& config)`: Tworzy sieć od zera na podstawie dostarczonej konfiguracji (`NetworkConfig`), inicjalizując warstwy, funkcje straty i `learningRate`.
- **Metody:**
    - `addLayer()`: Dodaje warstwy i aktualizuje wewnętrzny `NetworkConfig` (wykorzystywany do zapisu).
    - `forwardPass()` / `backwardPass()`: Koordynują propagację w przód i wstecz przez wszystkie warstwy.
    - `updateParameters()`: Koordynuje aktualizację wag w każdej warstwie (prosty SGD).
    - **`save()`:** Zapisuje kompletną sieć:
        - Konfiguracja sieci (`NetworkConfig`) jest zapisywana do pliku `config.json` (format JSON, z wcięciami). W tym pliku zapisywany jest również używany tryb mnożenia macierzy.
        - Wagi i biasy każdej warstwy są zapisywane do osobnych plików CSV (np. `layer_0_weights.csv`).
        - Implementuje **atomowy zapis** (zapis do plików tymczasowych, a następnie ich zamiana na finalne pliki), co chroni przed uszkodzeniem danych w przypadku awarii.
    - **`load()`:** Wczytuje sieć:
        - Odczytuje plik `config.json` i deserializuje go do `NetworkConfig`.
        - **Kluczowe:** Na podstawie wczytanego `NetworkConfig`, `load()` **samodzielnie buduje całą strukturę warstw sieci (`layers.clear()` i ponowne `addLayer()`).**
        - Odczytuje wagi i biasy z plików CSV i wypełnia nimi nowo utworzone warstwy.
        - Ustawia globalny tryb mnożenia macierzy (`Matrix::setMultiplicationMode()`) na podstawie wczytanej konfiguracji.
        - Implementuje solidną obsługę błędów podczas czytania plików i parsowania liczb.

Klasa Trainer<T>

- **Rola:** Koordynator procesu treningu.
- **Metody:**
  - `runTrainingLoop()`: Główna pętla treningowa. Iteruje przez epoki, tasuje dane, pobiera batche, wywołuje `forwardPass`, oblicza stratę, wywołuje `backwardPass` i `updateParameters`. Raportuje postęp (stratę, dokładność walidacyjną) i zapisuje historię do globalnych wektorów.
  - `validate()`: Wykonuje walidację modelu na zbiorze testowym.

Klasa Metrics<T>

- **Przeznaczenie:** Statyczna klasa do obliczania metryk wydajności.
- **Metody:** `computeAccuracy()` (oblicza dokładność, porównując przewidywane klasy z prawdziwymi etykietami).

Klasa User i UserManager

- **`User`:** Prosta struktura przechowująca nazwę użytkownika i hasło (uwaga: w tym projekcie hasła są przechowywane jako zwykły tekst, co jest niezalecane w prawdziwych aplikacjach ze względów bezpieczeństwa).
- **`UserManager`:** Zarządza bazą użytkowników (plik `users.dat`).
  - **Metody:** `registerUser()`, `loginUser()`. Hasła są maskowane gwiazdkami podczas wpisywania.
  - `getUserModelPath()`: Zwraca ścieżkę do katalogu, gdzie dany użytkownik może zapisywać swoje modele (np. `models/nazwa_uzytkownika/`).

Klasa Utils i Globals

- **`Utils.h/.cpp`:** Zawiera zbiór globalnych funkcji pomocniczych, które nie pasują do konkretnych klas:
  - `clearConsole()`, `pressAnyKeyToContinue()`, `readPasswordHidden()` (mas kowanie hasła).
  - Funkcje do integracji z SFML (`convertSfImageToMatrix`, `drawSquare`, `runSFMLMode`).
  - Funkcje do integracji z Gnuplotem (`plotTrainingHistory`).
  - Definicje globalnych wektorów `trainingLossHistory` i `validationAccuracyHistory`.
- **`Globals.h/.cpp`:** Przechowuje definicje globalnych stałych konfiguracyjnych projektu (np. rozmiary obrazów MNIST, ścieżki do plików danych, rozmiary okna SFML).

Interfejs Użytkownika

Interfejs Konsolowy (Panel Sterowania)

- **Panel Uwierzytelniania:** Pierwsze, co widzi użytkownik. Pozwala na zalogowanie się lub rejestrację.
- **Główny Panel Sterowania:** Dostępny po zalogowaniu. Umożliwia pełną kontrolę nad siecią neuronową poprzez wybór opcji z menu. Wszystkie wejścia użytkownika są walidowane, aby zapobiec błędom. Komunikaty są wyświetlane w konsoli.

Interfejs Graficzny (SFML Mode)

- **Okno rysowania:** Umożliwia użytkownikowi rysowanie cyfr (białą barwą na czarnym tle) myszką.
- **Odgadywanie w czasie rzeczywistym:** Za każdym razem, gdy użytkownik rysuje lub wymazuje, sieć neuronowa wykonuje predykcję na bieżącym obrazie. Wynik (odgadnięta cyfra i prawdopodobieństwo) jest dynamicznie aktualizowany i wyświetlany w oknie SFML.
- **Klawiszologia:** Spacja wymusza odświeżenie predykcji, 'C' czyści płótno.

Wyzwania i Problemy Napotkane Podczas Projektu

Typowe Błędy i Problemy

- **Wielokrotne definicje (`LNK2005`):** Najczęstszy problem. Wynikał z naruszenia Zasady Jednej Definicji (ODR), gdzie globalne zmienne lub funkcje były definiowane w więcej niż jednym pliku `.cpp`. Rozwiązano poprzez rygorystyczne rozdzielenie deklaracji (`extern` w `.h`) od definicji (tylko w jednym `.cpp`).
- **Konflikty makr (`std::max`, `std::log`):** Nagłówki systemu (`windows.h`) kolidowały z nazwami funkcji w `std::`. Rozwiązano poprzez użycie makra `NOMINMAX` lub jawne otoczenie wywołań funkcji nawiasami `(std::funkcja)`.
- **Integracja NVIDIA CUDA/cuBLAS:**
  - **`CUBLAS_STATUS_INVALID_VALUE` (`DGEMM`/`DGEAM` `parameter illegal value`):** Najtrudniejszy do rozwiązania problem. Wynikał z niezgodności między `row-major` (C++) a `column-major` (cuBLAS) i błędów w parametrach `cublasDgemm`/`cublasDgeam` (`m, n, k, lda, ldb, ldc`, `transa`, `transb`). Rozwiązano przez fizyczną transpozycję macierzy na GPU (`cublasDgeam`) przed i po mnożeniu (`cublasDgemm`), co okazało się najbardziej niezawodne.
- **Niskie wykorzystanie GPU (np. 24%):** Wskazywało na wąskie gardło po stronie CPU (CPU bottleneck).
  - **Rozwiązanie:** Zwiększenie `BATCH_SIZE_TRAIN` oraz zrównoleglenie operacji na CPU w `DataLoader` i `Layer`(dodawanie biasów, sumowanie gradientów biasów) za pomocą `std::thread`/`std::async`.
- **Sieć się nie uczy przy dużym batchu (overfitting/utknięcie):** Zbyt duży `BATCH_SIZE_TRAIN` (np. 2048) spowalniał konwergencję.
  - **Rozwiązanie:** Dobór optymalnego `BATCH_SIZE_TRAIN` (np. 128-512) i odpowiednie dostosowanie `learningRate`oraz liczby epok.
- **Problem z `std::stod` w `load()`:** Błędy parsowania liczb w plikach CSV.
  - **Rozwiązanie:** Dodano bloki `try-catch` dla `std::invalid_argument` i `std::out_of_range` podczas parsowania wartości.
- **Zarządzanie plikami i katalogami:** Błędy dostępu (`Cannot open file`), tworzenie katalogów.
  - **Wiedza:** `std::filesystem` jest kluczowe. Implementacja atomowego zapisu (`save()`) za pomocą plików tymczasowych (`.tmp`) i `fs::rename` w celu zapewnienia integralności danych.

Problemy ze Wsparciem dla Modułów C++20

- **Charakterystyka problemu:** Początkowo planowano wykorzystać funkcjonalność modułów C++20 (`import`, `module`) do organizacji kodu źródłowego, co miało na celu usprawnienie kompilacji i zarządzania zależnościami, a także poprawę czystości nagłówków. Jednakże, próby implementacji z użyciem modułów szybko napotkały na znaczące trudności.
- **Objawy:** Środowisko deweloperskie (Visual Studio 2022) zgłaszało liczne, niezrozumiałe błędy kompilacji (`fatal error`, `syntax error`, `cannot open source file` itp.), które nie były bezpośrednio związane z logiką kodu, lecz z wewnętrznymi mechanizmami kompilatora obsługującymi moduły. Często występowały problemy z odnajdywaniem deklaracji, mimo prawidłowego zapisu składni modułów. Proces kompilacji stawał się niestabilny, spowalniał, a nawet w niektórych przypadkach całkowicie zawieszał kompilator.
- **Decyzja i konsekwencje:** Ze względu na uporczywość problemów, ich niejasną naturę oraz brak natychmiastowych rozwiązań, podjęto decyzję o **rezygnacji z użycia modułów C++20** w tym projekcie. Zamiast tego, powrócono do tradycyjnego systemu zarządzania plikami źródłowymi opartym na dyrektywach preprocesora (`#include`).

Uzyskana Wiedza i Doświadczenie

- **Głębokie zrozumienie C++:** Praktyczne stosowanie zaawansowanych cech języka (szablony, RAII z `unique_ptr`, programowanie współbieżne, obsługa wyjątków, lambdy, `std::optional`).
- **Programowanie Systemowe i Niskopoziomowe:** Dogłębne zrozumienie zmiennych środowiskowych (`PATH`), konfiguracji projektów w środowiskach IDE (Visual Studio, `.vcxproj`), zarządzanie plikami (`std::filesystem`), interakcja z konsolą.
- **Programowanie Heterogeniczne (CPU/GPU):** Praktyczne wykorzystanie CUDA i cuBLAS. Rozumienie `row-major` vs `column-major`, optymalizacji `GEMM`, alokacji pamięci GPU, transferu danych. Zrozumienie, kiedy używać GPU, a kiedy CPU.
- **Architektura Sieci Neuronowych:** Praktyczne doświadczenie w implementacji MLP od podstaw, rozumienie działania `forward`/`backward pass`, roli funkcji aktywacji/straty/optymalizatorów.
- **Inżynieria Oprogramowania:**
  - **Modułowość:** Organizacja kodu w klasy i pliki (`.h`/`.cpp`).
  - **Testowanie i Debugowanie:** Niezwykle cenne doświadczenie w debugowaniu skomplikowanych problemów kompilacji, linkowania i wykonania. Umiejętność analizowania komunikatów kompilatora i profilera.
  - **Zarządzanie danymi:** Implementacja serializacji (JSON, CSV) i deserializacji.
  - **UI/UX:** Tworzenie użytecznego interfejsu konsolowego i graficznego.
  - **Bezpieczeństwo:** Nawet w prostym projekcie, dostrzeżenie potrzeby zabezpieczania haseł i atomowego zapisu danych.

Podsumowanie

Projekt "Sieć Neuronowa do Rozpoznawania Cyfr" stanowi kompleksową implementację od podstaw wielowarstwowej sieci neuronowej w języku C++. Zrealizowano wszystkie kluczowe komponenty niezbędne do działania modelu uczenia maszynowego, począwszy od operacji algebry liniowej na macierzach, poprzez mechanizmy ładowania i przetwarzania danych MNIST, aż po algorytm propagacji wstecznej. Szczególny nacisk położono na optymalizację wydajności, efektywnie wykorzystując wielowątkowość CPU oraz akcelerację GPU za pośrednictwem technologii NVIDIA CUDA i biblioteki cuBLAS dla operacji mnożenia macierzy.

Projekt rozbudowano o interaktywny interfejs konsolowy umożliwiający elastyczną konfigurację, trening, zapis i odczyt modeli, a także o graficzny tryb SFML do dynamicznego rozpoznawania rysowanych cyfr. Wprowadzono system zarządzania użytkownikami, co pozwala na personalizację zapisywanych modeli i historii treningu. Pomimo napotkanych znaczących wyzwań, takich jak problemy ze wsparciem dla modułów C++20, złożonością integracji CUDA/cuBLAS w środowisku Visual Studio i subtelnościami API SFML 3.0, projekt został pomyślnie ukończony.

Doświadczenie zdobyte podczas rozwiązywania tych problemów dostarczyło głębokiej wiedzy praktycznej z zakresu zaawansowanego programowania w C++, inżynierii oprogramowania, programowania heterogenicznego oraz fundamentalnych aspektów głębokiego uczenia. Projekt demonstruje zdolność do budowania wydajnych i złożonych systemów obliczeniowych od podstaw.

Możliwość Dalszego Rozwoju

Projekt, choć w pełni funkcjonalny i kompleksowy, stanowi solidną podstawę, którą można rozbudować o szereg zaawansowanych funkcji i optymalizacji, aby zwiększyć jego możliwości, wydajność i wszechstronność. Poniżej przedstawiono kluczowe kierunki dalszego rozwoju:

Implementacja Zaawansowanych Optymalizatorów

- Obecnie sieć wykorzystuje prosty algorytm Stochastic Gradient Descent (SGD). Dodanie wsparcia dla optymalizatorów takich jak **Adam**, **RMSprop**, **Adagrad**, czy **Nesterov Accelerated Gradient (NAG)**, może znacząco przyspieszyć proces treningu i poprawić zdolność sieci do znalezienia lepszych minimów funkcji straty. Wymagałoby to stworzenia hierarchii klas `Optimizer` i `AdamOptimizer`.

Techniki Regularyzacji

- Wprowadzenie metod regularyzacji, takich jak **Dropout** (losowe "wyłączanie" neuronów podczas treningu) oraz **L1/L2 Regularization** (dodawanie kar do funkcji straty za duże wagi), pomoże zapobiegać **przetrenowaniu (overfitting)**, szczególnie w przypadku bardziej złożonych sieci lub mniejszych zbiorów danych, poprawiając zdolność modelu do generalizacji na nowe, niewidzialne dane.

Wsparcie dla Innych Typów Warstw

- Rozszerzenie funkcjonalności sieci o inne typy warstw, takie jak **warstwy konwolucyjne (Convolutional Layers)** i **warstwy pooling (Pooling Layers)**, jest niezbędne do budowy konwolucyjnych sieci neuronowych (CNN), które są standardem w zadaniach przetwarzania obrazów (np. bardziej zaawansowane rozpoznawanie obrazów).
- Dodanie **warstw rekurencyjnych (Recurrent Layers)**, takich jak LSTM (Long Short-Term Memory) lub GRU (Gated Recurrent Unit), otworzyłoby drogę do przetwarzania danych sekwencyjnych (np. tekstu, szeregów czasowych).

Ulepszone Zarządzanie Pamięcią GPU i Asynchroniczność

- Obecna implementacja przesyła dane między CPU a GPU dla każdej operacji mnożenia macierzy. Optymalizacja `Matrix` tak, aby była "GPU-aware" (tj. aby mogła przechowywać swoje dane bezpośrednio na GPU i wykonywać operacje element-po-elemencie oraz transpozycje bezpośrednio na GPU, bez kopiowania do/z CPU w każdej operacji), mogłaby znacznie zwiększyć wydajność.
- Wykorzystanie **strumieni CUDA (CUDA Streams)** do asynchronicznego przesyłania danych i wykonywania obliczeń na GPU, tak aby CPU mogło

przygotowywać następne zadania, podczas gdy GPU pracuje nad bieżącymi, zminimalizowałoby wąskie gardła CPU.

Serializacja i Deserializacja Całej Sieci (Rozszerzenie)

- Rozważenie bardziej zunifikowanego formatu pliku (np. jeden plik binarny lub JSON/HDF5) dla całej sieci, włącznie z jej strukturą, wagami, biasami, a nawet stanem optymalizatora, ułatwiłoby zarządzanie modelami.

Wsparcie dla Innych Typów Danych i Zbiory Danych

- Rozszerzenie `DataLoader` o możliwość wczytywania innych formatów obrazów (np. PNG, JPEG) za pomocą bibliotek takich jak OpenCV lub niestandardowych formatów danych.
- Dodanie wsparcia dla innych popularnych zbiorów danych (np. CIFAR-10, Fashion MNIST).

Zwiększona Odporność na Błędy i Logowanie

- Rozbudowany system logowania wewnętrznych błędów, ostrzeżeń i informacji diagnostycznych do plików, co ułatwiłoby debugowanie i monitorowanie działania sieci w środowiskach produkcyjnych.
- Wprowadzenie bardziej zaawansowanych mechanizmów odzyskiwania po błędach w operacjach plikowych.

Wizualizacje w Czasie Rzeczywistym

- Rozbudowa interfejsu SFML o dynamiczne wizualizacje aktywacji warstw, map cech, lub funkcji straty w czasie rzeczywistym podczas treningu.

Ulepszone Zarządzanie Użytkownikami i Bezpieczeństwo

- Wprowadzenie kryptograficznie bezpiecznych algorytmów haszowania haseł w `UserManager`, aby zabezpieczyć dane użytkowników.
- Dodanie funkcjonalności zarządzania sesjami użytkowników i uprawnieniami.

Pakowanie w Uniwersalną Bibliotekę i Interfejsy dla Innych Języków

- Przekształcenie całego kodu w **uniwersalną bibliotekę**, która mogłaby być łatwo dołączana do innych projektów C++.
- Napisanie projektu **również** dla innych popularnych języków programowania, takich jak **Python**, **Java**, **Rust**, **Julia**, **Lua** czy **C#**.

Wdrożenie Testów Jednostkowych i Mutacyjnych

- **Testy jednostkowe:** Stworzenie rozbudowanego zestawu testów jednostkowych dla każdej klasy i kluczowych funkcji (np. operacji

macierzowych, funkcji aktywacji, propagacji w przód/wstecz, operacji DataLoader). Zapewniłoby to wysoką jakość kodu, ułatwiło refaktoryzację i wykrywanie błędów na wczesnym etapie rozwoju. Można wykorzystać frameworki takie jak Google Test.

- **Testy mutacyjne:** Bardziej zaawansowana forma testowania, która ocenia jakość testów jednostkowych poprzez wprowadzanie drobnych "mutacji" (błędów) w kodzie i sprawdzanie, czy istniejące testy jednostkowe są w stanie te mutacje wykryć. Zwiększyłoby to pewność, że testy są kompleksowe i skuteczne.

Integracja z Uczenia ze Wzmocnieniem (Reinforcement Learning) do Zastosowań w Grach

- Rozszerzenie projektu o możliwość wykorzystania sieci neuronowych w kontekście uczenia ze wzmocnieniem (RL). Pozwoliłoby to sieci na uczenie się optymalnych strategii działania w dynamicznym środowisku (np. w grach) poprzez interakcję i otrzymywanie nagród/kar.

Linki do Używanych Bibliotek

Poniżej znajdują się oficjalne strony internetowe lub repozytoria GitHub bibliotek zewnętrznych wykorzystanych w projekcie:

1. **SFML (Simple and Fast Multimedia Library)**
   - **Opis:** Wieloplatformowa biblioteka C++ do tworzenia gier i aplikacji multimedialnych. Używana w projekcie do tworzenia okna graficznego, rysowania i obsługi zdarzeń.
   - **Link:** https://www.sfml-dev.org/download/sfml/3.0.0/
   - **GitHub:** https://github.com/SFML/SFML
2. **NVIDIA CUDA Toolkit**
   - **Opis:** Zestaw narzędzi deweloperskich i bibliotek stworzonych przez NVIDIA do programowania GPU. Umożliwia pisanie kodu (CUDA C/C++) wykonywanego bezpośrednio na procesorach graficznych NVIDIA.
   - **Link:** https://developer.nvidia.com/cuda-downloads
3. **NVIDIA cuBLAS**
   - **Opis:** Biblioteka Basic Linear Algebra Subprograms (BLAS) firmy NVIDIA, zoptymalizowana pod kątem GPU. Jest częścią NVIDIA CUDA Toolkit i służy do wysoce wydajnego mnożenia macierzy i innych operacji liniowo-algebraicznych.
   - **Link:** Dołączona do CUDA Toolkit.
     Dokumentacja: https://docs.nvidia.com/cuda/cublas/
4. **nlohmann/json (JSON for Modern C++)**
   - **Opis:** Popularna, nagłówkowa biblioteka C++ do łatwej pracy z formatem danych JSON (serializacja i deserializacja).
   - **Link:** https://github.com/nlohmann/json
5. **Gnuplot**
   - **Opis:** Wolne, przenośne narzędzie do rysowania wykresów danych z linii komend. Wykorzystane do wizualizacji historii treningu sieci.
   - **Link:** http://www.gnuplot.info/